Incident report: payments-api latency from one merchant’s retry loop

Document: INC-304

Date: 2026-03-03

Services: payments-api

Summary

On March 3, 2026, merchant m_77812 experienced integration issues that caused their client to retry failed requests in a tight loop. This retry behavior generated excessive load on payments-api, raising p99 latency to 4.2 seconds across all merchants. The incident was classified as SEV3. Mitigation was achieved through manual rate-limit override. A permanent solution was implemented via CHG-1036.

Impact

All merchants using payments-api experienced elevated latency during the incident window. The p99 latency of 4.2 seconds degraded the payment experience for legitimate transactions. Merchant m_77812’s integration remained unable to process payments successfully due to their underlying client-side issue.

Timeline

- 14:23 UTC: Elevated latency detected on payments-api
- 14:31 UTC: On-call engineer identified merchant m_77812 as source of excessive request volume
- 14:45 UTC: Rate-limit override applied to merchant m_77812
- 14:52 UTC: p99 latency returned to normal levels
- 15:10 UTC: Merchant m_77812 contacted and advised to investigate their retry logic

Root Cause

Merchant m_77812’s integration was configured to retry failed requests without exponential backoff or jitter. When their client encountered transient errors, the retry loop generated requests at a rate that consumed disproportionate API capacity, impacting latency for all other merchants on the shared payments-api infrastructure.

Resolution

The on-call engineer applied a rate-limit override to throttle requests from merchant m_77812, immediately reducing their request volume and restoring normal latency for all merchants. This mitigation followed procedures documented in RB-02.

Follow-up Actions

CHG-1036 implemented a permanent per-merchant rate limit on payments-api to prevent similar incidents. This change ensures that any single merchant’s traffic spike cannot degrade service for others. Merchant m_77812 was advised to implement exponential backoff in their retry logic.